Figura 1: DevPixelForge

Rust Go [image: License: GPL v3] Platform

DevPixelForge (dpf)

High-performance multimedia processing engine in Rust with Go client for seamless integration.

«Transform pixels at the speed of Rust.»

What is DevPixelForge?

DevPixelForge (dpf) is a **high-performance multimedia processing engine** that provides:

Category	Operations
Images	Resize, crop, rotate, watermark, adjust, optimize, convert, palette, favicon, sprite, placeholder, srcset, EXIF
Documents	Markdown to PDF (GlyphWeaveForge + Typst)
Video	Transcode, resize, trim, thumbnail, web profiles
Audio	Transcode, trim, normalize (LUFS), silence removal

Key Features

- 🚀 **High Performance** — Rust-powered with parallel processing (Rayon)
- 🔄 **Multiple Formats** — PNG, JPEG, WebP, GIF, SVG, ICO, AVIF, MP4, WebM, MP3, AAC, Opus
- 📡 **Streaming Mode** — Persistent process for low-latency operations
- 🔗 **Go Integration** — Native FFI bindings via StreamClient
- 📄 **Markdown to PDF** — Typst-backed document rendering for file and inline flows
- 🎯 **Smart Operations** — Focal point cropping, auto-quality, entropy-based selection
- 📦 **Static Binary** — musl-compiled for portability

Markdown-to-PDF Highlights

- Uses GlyphWeaveForge `0.1.3` with the Typst backend selected explicitly.
- Resolves `glyphweaveforge` directly from crates.io; no local `[patch.crates-io]` override is required.
- Supports file input, inline text, or base64-encoded Markdown.
- Supports file output, directory output, and inline base64 PDF responses.
- Supports `resource_files` for in-memory Markdown inputs that reference local assets.
- Supports built-in GlyphWeaveForge themes including `engineering`, `professional`, `invoice`, `scientific_article`, and `informational`.

How It Works

text

```
┌──────────────────┐        JSON/stdio        ┌────────────────────┐
│   Go Bridge      │◄────────────────────────►│   dpf (Rust)       │
│  (Client)        │    stdin/stdout          │  (Engine)          │
└──────────────────┘                          └────────────────────┘
```

1. **Send Job** — JSON job definition via stdin
2. **Process** — Rust engine handles the operation
3. **Receive Result** — JSON response via stdout

Quick Start

Installation

bash

```bash
# Clone and build
git clone https://github.com/your-org/devpixelforge.git
cd devpixelforge
make build

# Verify capabilities
./dpf/target/release/dpf caps
```

Basic Usage

bash

```bash
# Single operation (one-shot)
./dpf/target/release/dpf process \
  --job '{"operation":"resize","input":"image.png","output_dir":"out","widths":[320,640]}'

# Batch processing
./dpf/target/release/dpf batch --file jobs.json

# Markdown to PDF
./dpf/target/release/dpf process \
  --job '{"operation":"markdown_to_pdf","markdown_text":"# Hello\n\nPDF","inline":true}'

# Streaming mode (persistent process)
./dpf/target/release/dpf --stream
```

Usage Modes

Mode	Command	Use Case
One-shot	`dpf process --job '{...}'`	Single operation
Stdin	`echo '{...}' | dpf`	Pipes and scripts
Streaming	`dpf --stream`	Multiple operations, low latency
Batch	`dpf batch --file jobs.json`	Parallel job processing

Streaming Mode Example

bash

```bash
# Start streaming process
./dpf/target/release/dpf --stream

# Send multiple jobs (one per line)
{"operation":"resize","input":"a.png","output_dir":"out","widths":[320]}
{"operation":"optimize","inputs":["b.png"],"output_dir":"out"}
{"operation":"watermark","input":"c.png","output":"out/c.png","text":"© 2024"}
```

Operations Reference

Image Operations

Operation	Description	Example
`resize`	Resize to multiple widths	`{"widths":[320,640,1024]}`
`crop`	Manual or smart crop	`{"gravity":"focal_point","focal_x":0.75}`
`rotate`	Rotate/flip	`{"angle":90}`
`watermark`	Text/image overlay	`{"text":"© 2024","position":"bottom-right"}`
`adjust`	Brightness, contrast, blur	`{"brightness":0.2,"blur":2.0}`
`quality`	Auto-quality optimization	`{"target_size":50000}`
`srcset`	Responsive images + HTML	`{"widths":[320,640],"generate_html":true}`
`exif`	Strip/extract EXIF	`{"exif_op":"strip","mode":"all"}`
`optimize`	Lossless/lossy compression	`{"level":"lossless"}`
`convert`	Format conversion	`{"format":"webp"}`
`palette`	Color reduction	`{"max_colors":32,"dithering":0.5}`
`favicon`	Multi-size favicons	`{"sizes":[16,32,180]}`
`sprite`	Sprite sheets	`{"inputs":["a.png","b.png"],"columns":2}`
`placeholder`	LQIP, dominant color	`{"kind":"lqip","width":20}`
`markdown_to_pdf`	Render Markdown as PDF	`{"markdown_text":"# Report","inline":true}`

Document Operation

`markdown_to_pdf` accepts exactly one source from `input`, `markdown_text`, or `markdown_base64` and at least one output mode from `output`, `output_dir`, or `inline`.

- File input + `output` / `output_dir` keeps filesystem-relative asset resolution.
- Inline input + `inline=true` returns base64 PDF bytes in `outputs[0].data_base64`.
- Inline input + `resource_files` injects href-to-file asset mappings through GlyphWeaveForge's public resource resolver.
- Inline input + `output_dir` requires `file_name`.
- Built-in themes validated locally: `invoice`, `scientific_article`, `professional`, `engineering`, `informational`.

Example:

```json
{
  "operation": "markdown_to_pdf",
  "markdown_text": "# Report\n\nRendered by Typst.",
```

```
  "inline": true,
  "theme": "professional",
  "resource_files": {
    "logo.png": "./assets/logo.png"
  }
}
```

CLI Validation Artifacts

The repository includes a reproducible CLI validation guide and generated sample PDFs for this feature:

- `docs/validation/markdown-to-pdf/readme.pdf`
- `docs/validation/markdown-to-pdf/agents.pdf`
- `docs/validation/markdown-to-pdf/themes/`

Video Operations

Operation	Description	Example
`transcode`	Codec conversion	`{"codec":"h264"}`
`resize`	Scale video	`{"height":720}`
`trim`	Cut by timestamps	`{"start":30,"end":90}`
`thumbnail`	Extract frame	`{"timestamp":"25%"}`
`profile`	Web-optimized	`{"profile":"web-mid"}`

Audio Operations

Operation	Description	Example
`transcode`	Format conversion	`{"codec":"aac","bitrate":"192k"}`
`trim`	Cut by timestamps	`{"start":0,"end":60}`
`normalize`	LUFS normalization	`{"target_lufs":-14}`
`silence_trim`	Remove silence	`{"threshold_db":-40}`

Project Structure

text

```
devpixelforge/
├── dpf/                          # Rust Engine
│   └── src/
│       ├── lib.rs                # Main entry
│       ├── cli.rs                # CLI arguments
│       ├── processor.rs           # Job processing
│       └── operations/
│           ├── image/             # 14 image operations
│           ├── video/             # 5 video operations
```

```
│           └── audio/              # 4 audio operations
├── go-bridge/                     # Go FFI Bindings
│   └── pkg/dpf/
│       ├── client.go              # ProcessClient
│       └── stream.go              # StreamClient
├── docs/                          # Documentation
│   ├── README.md                  # This file (index)
│   ├── schema.md                  # JSON schema reference
│   ├── examples.md                # Working examples
│   └── testing/                   # Testing docs
└── Makefile
```

Building

Requirements

Dependency	Version	Purpose
Rust	≥ 1.74	Core engine
Go	≥ 1.21	FFI bridge
musl-tools	(Debian/Ubuntu)	Static binary

Build Commands

bash

```bash
# Full build (Rust + Go)
make build

# Rust only
make build-rust

# Go only
make build-go

# Static binary (musl)
make build-rust-static
```

Testing

bash

```bash
# All tests
make test

# Rust tests
cd dpf && cargo test

# Go tests
cd go-bridge && go test -v
```

Component	Tests
Rust Operations	280+
Integration	20+
Go Bridge	16+
Total	**316+**

Documentation

Document	Description
📖 Main Docs	Complete project documentation
📋 JSON Schema	Full JSON protocol reference
💡 Examples	Working examples for all operations
🧪 Testing	Testing architecture and guides
✅ Markdown-to-PDF Validation	Reproduction steps and committed validation artifacts

---

License

GNU General Public License v3.0 (GPL-3.0)

Copyright (c) 2024 Ing. Gustavo Gutiérrez

This program is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program. If not, see https://www.gnu.org/licenses/.